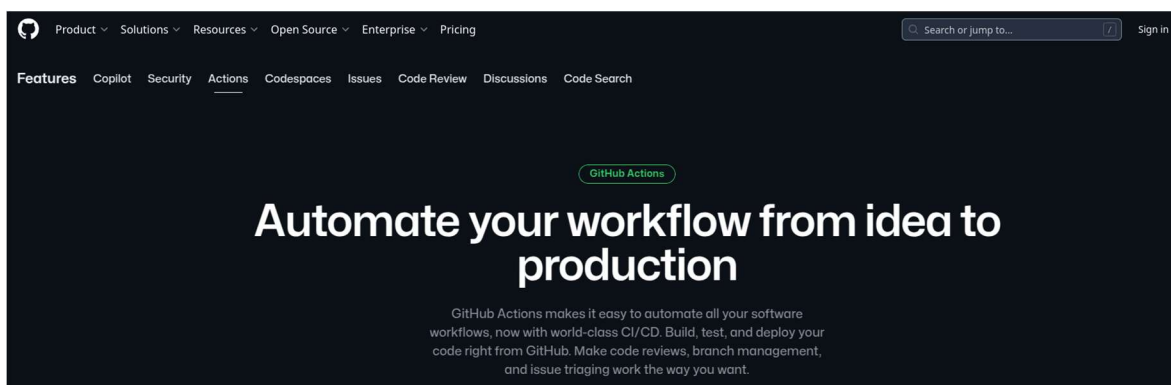


Using Web Proxy and Github Actions with Docker

1. Overview



In this lab, you are going to learn how to set up a Web Proxy and use Github Actions, together with docker based services.

GitHub Actions is a CI/CD (Continuous Integration and Continuous Deployment) automation tool built into GitHub. In your project, you will be using Github repository and Github Actions to help with automated build, testing and deployment of your project.

2. Outcomes

Upon completion of this session, you should be able to:

- Install and run a web server/proxy (Nginx) using Docker locally at your laptop

- Obtain a valid free CA cert from Let's Encrypt for your Web Proxy
- Create Github Actions CI/CD pipeline to build, test and deliver a sample web application
- Configure your team GitHub repository, Github Actions, and docker services at your team VM to commence CI/CD implementation phase for your team project

3. Installation

To get you familiar and comfortable with Docker which is a valuable skill in the industry, we will continue the use of Docker in this Lab exercise.

Pre-requisite: Docker

Install docker-compose. The steps are found in lab 1.

To test that your Docker is installed successfully, open a terminal and run the hello-world Docker image:

```
$ docker run hello-world

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
ca4f61b1923c: Pull complete
Digest: sha256:ca0eeb6fb05351dfc8759c20733c91def84cb8007aa89a5bf606bc8b315b9fc7
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

A list of useful Docker CLI commands is available at:

<https://dockerlabs.collabnix.com/docker/cheatsheet/>

Pre-requisite: Docker Compose

Install docker-compose. The steps are found in lab 1.

Co-requisite: Git

We will use Git as the Source Code Management (SCM) tool, and also Github and Github Actions, to build and test your codes whenever there are changes.

In case you need to re-install or update Git, go to <https://git-scm.com/downloads> to download a copy for your OS platform. Similar as Docker, you can open a terminal to run Git commands. A list of useful commands is available at

<https://training.github.com/downloads/github-git-cheat-sheet.pdf>

4. Tutorial

To understand the use of CI/CD for secure web application development, you will now try using Github Actions to automate the process of building, testing and delivering of a sample web application. This process of building, testing and delivering is going to be repetitive during the implementation phase which explains why CI/CD is popular in the industry.

Your web application is usually served behind a webproxy, which also handles the TLS/SSL termination.

4.1. A Web Server, Nginx, using Docker

First, we will set up a webserver and configure it as a proxy to show a simple HTML hello world page.

Create a directory for your Nginx setup:

```
mkdir -p ./nginx-proxy && cd ./nginx-proxy
```

Copy the provided Nginx Docker Compose file (nginx-compose.yml) file to the *nginx-proxy* directory.

```
cp ./nginx-compose.yml ./index.html ./nginx-proxy/
```

You should review and understand the content of the provided lab files.

4.2. Configure Nginx as a Reverse Proxy

Copy the provided Nginx configuration file (nginx.conf) to the *nginx-proxy* directory.

```
cp ./nginx.conf ./nginx-proxy/
```

Please update the yourdomain.com to your project domain.

Start the services using docker-compose

```
docker-compose -f nginx-compose.yml up
```

```
curl http://localhost/
```

Or open the above link in a web browser.

4.3. Configure Let's Encrypt TLS / SSL with Certbot

Now that you have successfully configure a web proxy to serve a simple HTML page, we will modify the configuration files to add TLS / SSL to the web proxy. In this lab, we will be using Let's Encrypt (Certificate Authority) and Certbot.

The Certificate Authority, CA, (Let's Encrypt) will need to verify domain ownership before issuing your cert. Let's Encrypt supports 2 commonly used domain verification methods:

1. ACME challenge. Let's Encrypt gives a token to your ACME client (Certbot), and your ACME client puts a file on your web server at `http://<YOUR_DOMAIN>/.well-known/acme-challenge/<TOKEN>`.
2. DNS challenge. This challenge asks you to put a specific value in a TXT record under your domain name.

Copy the provided Docker Compose file (`nginx-compose-tls.yml`) to the *nginx-proxy* directory. It added certbot and let's encrypt configuration. Please note that: Certbot must be run with `certonly` mode to obtain the certificate.

If you are able to configure the network to use ACME challenge, you can run certbot command in its container or add an entrypoint configuration to do this.

Please update the "email@example.com" and "yourdomain.com" to what your project is using.

```
entrypoint: sh -c "certbot certonly --webroot -w /var/www/certbot --email
your-email@example.com --agree-tos --no-eff-email --rsa-key-size 4096 --
expand --force-renewal -d yourdomain.com -d www.yourdomain.com && nginx -s
reload"
```

If you do not have access to network configuration, you can use the DNS challenge instead.

```
sudo certbot certonly --manual --preferred-challenges dns -d yourdomain.com
-d "www.yourdomain.com" --agree-tos -m your-email@example.com -no-eff-email
```

You will need to put TXT string into the DNS associated with your domain.

After obtaining the cert, restart `nginx` so it loads the new certificates.

4.4. Nginx should listen on HTTPS (443)

Configure Nginx to listen to port 443. If you are using ACME challenge, you should add port 80:80 in Nginx service. Certbot uses HTTP to verify domain ownership (via `/.well-known/acme-challenge/`).

4.5. Modify the provided nginx configuration file (nginx-tls.conf)

Modify the provided nginx configuration template according to your project needs.

The `ssl_certificate` and `ssl_certificate_key` should be referenced in `nginx.conf`.

Copy your configuration files to `./nginx-proxy` directory

```
cp ./nginx-compose-tls.yml ./nginx-proxy/nginx-compose.yml
cp ./nginx-tls.conf ./nginx-proxy/nginx.conf
```

```
docker-compose -f nginx-compose.yml up
```

To test:

```
curl https://yourdomain.com/
```

Or open the above link in a web browser.

4.6. Auto-Renewal for TLS / SSL Certificates (Optional)

If you are using ACME challenge, you can set up automatic renewal of Let's Encrypt certificates. Automating DNS challenge will require setting the right access to your DNS.

Let's Encrypt certificates expire every 90 days. Add a cron job for automatic renewal:

```
crontab -e

0 3 * * * docker compose -f certbot-compose.yml run --rm certbot renew &&
docker compose restart nginx
```

This will renew SSL certificates every 3 AM daily.

5. Github Actions and GitHub

Github Actions is a useful CI/CD workflow tool, for Github repositories.

The configuration files are located at `.github/workflows/` folder.

A sample configurations is provided as `test-nginx.yml`. Please place the file in your `.github/workflows/` folder.

We will continue to use Github Actions in subsequent labs. Please take some time to be familiar with Github Actions workflows syntax.

6. Team Project

You should apply what you have learnt in this lab on your team project. A reverse proxy with TLS / SSL termination is important to secure your web application. A CI/CD tool enables automated build, testing and deployment of your application, which significantly reduce the security risks of the web app if the test cases are well written.

7. References

<https://letsencrypt.org/how-it-works/>

<https://datatracker.ietf.org/doc/html/rfc8555>

<https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions>

END OF DOCUMENT